

Lecture 03

Memory-based Recommender Systems (part 1)

Euhyun Moon, Ph.D.

Machine Learning System (MLSys) Lab

Computer Science and Engineering

Sogang University



Today's Class

- Memory-based Recommender Systems
 - User-based Neighborhood Methods
 - Item-based Neighborhood Methods



Memory-based Recommender Systems

- Memory-based recommender systems were among the earliest algorithms developed for collaborative filtering
- These algorithms are based on the premise that **similar users exhibit similar rating behavior patterns**, and **similar items receive similar ratings**
- There are two types of memory-based algorithms: user-based and item-based neighborhood methods

Memory-based Recommender Systems

- **User-based neighborhood method**

- The ratings provided by users similar to a **target user** u_i , are used to make recommendations for u_i
- The predicted ratings for a user u_i are computed as the weighted average of the “**peer group**” ratings for each item

- **Item-based neighborhood method**

- For a **target item** v_j , the first step is to determine a set S_j of items that are most similar to item v_j
- To predict the rating of a target user u_i for an item, the ratings in set S_j specified by u_i are determined
- The weighted average of these ratings is used to compute the predicted rating of user u_i for item v_j

Memory-based Recommender Systems

- **Important Distinction between Two Approaches**

- In the former case, the ratings are predicted based on the ratings of neighboring users
- In the latter case, the ratings are predicted based on the user's own ratings of neighboring items
- In the former case, neighborhoods are defined by similarities among users (the rows of the rating matrix)
- In the latter case, neighborhoods are defined by similarities among items (the columns of the rating matrix)
- Thus, the two methods have a complementary relationship

	v_1	v_2	v_3	...	v_m	
u_1	2			3	3	
u_2		5	5			5
u_3		4	3			
u_4	5				4	
u_5		2		3		4
...	4		5			
			1	1		
		2			3	
u_n	2			4		5

<User-Item Interaction Matrix>



Predicting Ratings with Neighborhood-based Methods

Concept of Neighborhood-based Methods

- The basic idea behind neighborhood-based methods is to use either user-user similarity or item-item similarity to make recommendations from a rating matrix
- The concept of a **neighborhood** implies that either **similar users or similar items must be determined** to make predictions
- We will discuss how neighborhood-based methods can be used to predict ratings for specific user-item combinations

User-based Neighborhood Methods

- User-based neighborhoods are defined to identify users similar to the **target user** for whom rating predictions are being computed
- To determine the neighborhood of the target user u_i , the similarity between u_i and all other users is computed
- Therefore, a similarity function must be defined based on the ratings provided by users

User-based Neighborhood Methods

- Computing similarity can be challenging because different users may use different rating scales
- One user might be biased toward liking most items, whereas another user might be biased toward disliking most items
- Furthermore, different users may have rated different sets of items
- Therefore, mechanisms must be developed to address these issues

User Similarity

- For an $n \times m$ rating matrix $R = [r_{uv}]$ with n users and m items, let I_u denote the set of item indices for which user u (a row) has provided ratings
- For example,
 - $I_2 = \{v_2, v_3, \dots, v_m\}$
 - $I_3 = \{v_2, v_3, \dots\}$
- The set of items rated by both users u_i and u_j is given by $I_i \cap I_j$
- For example,
 - $I_2 \cap I_3 = \{v_2, v_3, \dots, v_m\} \cap \{v_2, v_3, \dots\} = \{v_2, v_3\}$

	v_1	v_2	v_3	...	v_m	
u_1	2			3	3	
u_2		5	5			5
		4	3			
u_3						
u_4	5				4	
u_5		2		3		4
...	4		5			
			1	1		
		2			3	
u_n	2			4		5

<User-Item Interaction Matrix R >

User Similarity

- The set $I_i \cap I_j$ defines the mutually observed ratings, which are used to compute the similarity between the i -th and j -th users for neighborhood computation
- It is possible, and quite common, for $I_i \cap I_j$ to be an **empty set** because ratings matrices are typically sparse

User Similarity

- One measure that captures the similarity $Sim(u_i, u_j)$ between the rating vectors of two users, u_i and u_j , is the **Pearson correlation coefficient**
- The first step is to compute the mean rating μ for each user based on their specified ratings, as follows:

$$\mu_u = \frac{\sum_{k \in I_u} r_{uk}}{|I_u|} \quad \forall u \in \{1, 2, \dots, m\}$$

- Then, the Pearson correlation coefficient between the rows u_i and u_j is defined as follows:

$$Sim(u_i, u_j) = Pearson(u_i, u_j) = \frac{\sum_{v \in I_i \cap I_j} (r_{iv} - \mu_i) \cdot (r_{jv} - \mu_j)}{\sqrt{\sum_{v \in I_i \cap I_j} (r_{iv} - \mu_i)^2} \cdot \sqrt{\sum_{v \in I_i \cap I_j} (r_{jv} - \mu_j)^2}}$$

Mean-Centered Rating

- Different users may provide ratings on varying scales
- One user might rate all items highly, whereas another user might rate all items negatively
- Therefore, the raw ratings need to be mean-centered in a row-wise fashion

$$s_{uv} = r_{uv} - \mu_u \quad \forall u \in \{1, 2, \dots, n\}$$

Mean-Centered Rating Prediction

- The peer group $P_i(v)$ of user u_i is defined as the top- k users who have high similarity with u_i and have provided a rating for item v
- The average of the mean-centered ratings of an item within the top- k peer group of the target user u is used to provide a mean-centered prediction

$$\frac{\sum_{u_j \in P_i(v)} \text{Sim}(u_i, u_j) \cdot s_{jv}}{\sum_{u_j \in P_i(v)} |\text{Sim}(u_i, u_j)|}$$

- Then, the mean rating of the target user is added back to this prediction to provide a raw rating prediction $\hat{r}_{uv}$ for the target user u on item v :

$$\hat{r}_{iv} = \mu_i + \frac{\sum_{u_j \in P_i(v)} \text{Sim}(u_i, u_j) \cdot s_{jv}}{\sum_{u_j \in P_i(v)} |\text{Sim}(u_i, u_j)|} = \mu_i + \frac{\sum_{u_j \in P_i(v)} \text{Sim}(u_i, u_j) \cdot (r_{jv} - \mu_j)}{\sum_{u_j \in P_i(v)} |\text{Sim}(u_i, u_j)|}$$

Mean-Centered Rating Prediction

- Example
 - User similarity between u_1 and u_3

Table 2.1: User-user similarity computation between user 3 and other users

Item-Id $\Rightarrow$	1	2	3	4	5	6	Mean Rating	Cosine($i, 3$) (user-user)	Pearson($i, 3$) (user-user)
User-Id $\Downarrow$									
1	7	6	7	4	5	4	5.5	0.956	0.894
2	6	7	?	4	3	4	4.8	0.981	0.939
3	?	3	3	1	1	?	2	1.0	1.0
4	1	2	2	3	3	4	2.5	0.789	-1.0
5	1	?	1	2	3	3	2	0.645	-0.817

$$\text{Cosine}(u_1, u_3) = \frac{6 * 3 + 7 * 3 + 4 * 1 + 5 * 1}{\sqrt{6^2 + 7^2 + 4^2 + 5^2} \cdot \sqrt{3^2 + 3^2 + 1^2 + 1^2}} = 0.956$$

$$\text{Pearson}(u_1, u_3) = \frac{(6 - 5.5) * (3 - 2) + (7 - 5.5) * (3 - 2) + (4 - 5.5) * (1 - 2) + (5 - 5.5) * (1 - 2)}{\sqrt{0.5^2 + 1.5^2 + (-1.5)^2 + (-0.5)^2} \cdot \sqrt{1^2 + 1^2 + (-1)^2 + (-1)^2}} = 0.894$$

Mean-Centered Rating Prediction

- Example (cont'd)
 - The rating predictions for v_1 and v_6 by u_3 are derived from the top-2 peer group of u_1 and u_2

Table 2.1: User-user similarity computation between user 3 and other users

Item-Id $\Rightarrow$	1	2	3	4	5	6	Mean Rating	Cosine($i, 3$) (user-user)	Pearson($i, 3$) (user-user)
User-Id $\Downarrow$									
1	7	6	7	4	5	4	5.5	0.956	0.894
2	6	7	?	4	3	4	4.8	0.981	0.939
3	?	3	3	1	1	?	2	1.0	1.0
4	1	2	2	3	3	4	2.5	0.789	-1.0
5	1	?	1	2	3	3	2	0.645	-0.817

$$\hat{r}_{31} = \frac{0.894 * 7 + 0.939 * 6}{0.894 + 0.939} = 6.49$$

$$\hat{r}_{36} = \frac{0.894 * 4 + 0.939 * 4}{0.894 + 0.939} = 4$$

Mean-Centered Rating Prediction

- Example (cont'd)
 - The mean-centered rating prediction for u_3 provides a better relative prediction

Table 2.1: User-user similarity computation between user 3 and other users

Item-Id $\Rightarrow$	1	2	3	4	5	6	Mean Rating	Cosine($i, 3$) (user-user)	Pearson($i, 3$) (user-user)
User-Id $\Downarrow$									
1	7	6	7	4	5	4	5.5	0.956	0.894
2	6	7	?	4	3	4	4.8	0.981	0.939
3	?	3	3	1	1	?	2	1.0	1.0
4	1	2	2	3	3	4	2.5	0.789	-1.0
5	1	?	1	2	3	3	2	0.645	-0.817

$$\hat{r}_{31} = 2 + \frac{0.894 * (7 - 5.5) + 0.939 * (6 - 4.8)}{0.894 + 0.939} = 3.35$$

$$\hat{r}_{36} = 2 + \frac{0.894 * (4 - 5.5) + 0.939 * (4 - 4.8)}{0.894 + 0.939} = 0.86$$

Similarity Function Variants

- **Raw Cosine Similarity**

$$\text{Raw Consine}(u_i, u_j) = \frac{\sum_{v \in I_i \cap I_j} r_{iv} \cdot r_{jv}}{\sqrt{\sum_{v \in I_i \cap I_j} r_{iv}^2} \cdot \sqrt{\sum_{v \in I_i \cap I_j} r_{jv}^2}}$$

$$\text{Raw Consine}(u_i, u_j) = \frac{\sum_{v \in I_i \cap I_j} r_{iv} \cdot r_{jv}}{\sqrt{\sum_{v \in I_i} r_{iv}^2} \cdot \sqrt{\sum_{v \in I_j} r_{jv}^2}}$$

- In general, the Pearson correlation coefficient is preferred over the raw cosine similarity due to the **bias adjustment effect** of mean centering
- This adjustment accounts for the fact that different users exhibit varying levels of generosity in their global (overall) rating patterns

Similarity Function Variants

- **Significance Weighting**

- The reliability of the similarity function $Sim(u_i, u_j)$ is often affected by the number of common ratings, $|I_i \cap I_j|$, between users u_i and u_j
- When two users have only a small number of ratings in common, a discount factor should be applied to the similarity function to de-emphasize the importance of that user pair
- This method is known as **significance weighting**
- The discount factor is applied when the number of common ratings between the two users is less than a particular threshold β

$$DiscountedSim(u_i, u_v) = Sim(u_i, u_v) \cdot \frac{\min\{|I_i \cap I_j|, \beta\}}{\beta}$$

- The discounted similarity is used both in determining the peer group and in computing the rating prediction

Rating Prediction Function Variants

- **Z-Score**

- The Z-score z_{uv} further divides s_{uv} by the standard deviation σ_u of the observed ratings of user u

$$\sigma_u = \sqrt{\frac{\sum_{v \in I_u} (r_{uv} - \mu_u)^2}{|I_u| - 1}} \quad \forall u \in \{1, 2, \dots, n\}$$

- The standard ratings is computed as follows:

$$z_{uv} = \frac{r_{uv} - \mu_u}{\sigma_u} = \frac{s_{uv}}{\sigma_u}$$

- The predicted rating $\hat{r}_{uv}$ is modified as follows:

$$\hat{r}_{iv} = \mu_i + \sigma_i \frac{\sum_{u_j \in P_i(v)} \text{Sim}(u_i, u_j) \cdot z_{jv}}{\sum_{u_j \in P_i(v)} |\text{Sim}(u_i, u_j)|}$$

Variations in Filtering Peer Groups

- **Low and Negative Correlation**

- The simplest approach is to use the top- k most similar users to the target user as their peer group
- Such an approach might include users who are **weakly or negatively correlated** with the target user
- Weakly and negatively correlated users may increase the error in the prediction
- Therefore, ratings from users with weak or negative correlations are often filtered out

Impact of the Long Tail

- **Long-Tailed Distribution on Ratings**
 - Some items may be very popular and repeatedly appear as commonly rated items by different users
 - Such ratings can sometimes degrade the quality of recommendations because they tend to be **less discriminative** among different users

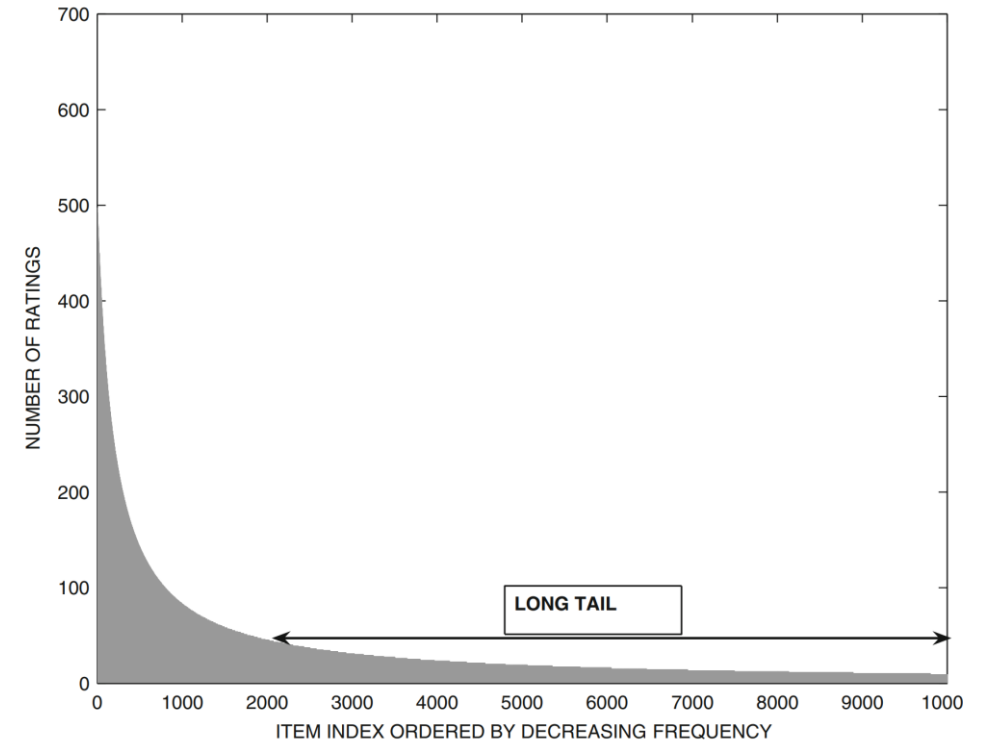


Figure 2.1: The long tail of rating frequencies

Impact of the Long Tail

- **Inverse Document Frequency (idf)**

- Apply a penalty to popular items and assign more weight to unique items
- Let w_v be the weight of item v and n_v be the number of ratings for item v

$$w_v = \log\left(\frac{n}{n_v}\right) \quad \forall v \in \{1, 2, \dots, m\}$$

- The Pearson correlation coefficient is modified using the weight as follows:

$$\text{Pearson}(u_i, u_j) = \frac{\sum_{v \in I_i \cap I_j} w_v \cdot (r_{iv} - \mu_i) \cdot (r_{jv} - \mu_j)}{\sqrt{\sum_{v \in I_i \cap I_j} w_v \cdot (r_{iv} - \mu_i)^2} \cdot \sqrt{\sum_{v \in I_i \cap I_j} w_v \cdot (r_{jv} - \mu_j)^2}}$$

Item-based Neighborhood Methods

- In item-based methods, **peer groups** are constructed based on **items** rather than **users**
- Similarities need to be computed between items, which correspond to the columns in the ratings matrix

Item-based Neighborhood Methods

- For an $n \times m$ rating matrix $R = [r_{uv}]$ with n users and m items, let U_v denote the set of user indices for which ratings have been specified for item v (a column)
- For example,
 - $U_2 = \{u_2, u_3, u_5, \dots\}$
 - $U_3 = \{u_2, u_3, \dots\}$
- The set of users who have provided ratings for both item v_i and v_j is given by $U_i \cap U_j$
- For example,
 - $U_2 \cap U_3 = \{u_2, u_3, u_5, \dots\} \cap \{u_2, u_3, \dots\} = \{u_2, u_3\}$

	v_1	v_2	v_3	...	v_m	
u_1	2			3	3	
u_2		5	5			5
u_3		4	3			
u_4	5				4	
u_5		2		3		4
...	4		5			
			1	1		
		2			3	
u_n	2			4		5

<User-Item Interaction Matrix R >

Adjusted Cosine Similarity

- The adjusted cosine similarity between two items, v_i and v_j , is computed using mean-centered ratings s_{uv} as follows:

$$\text{Adjusted Consine}(v_i, v_j) = \frac{\sum_{u \in U_i \cap U_j} s_{ui} \cdot s_{uj}}{\sqrt{\sum_{u \in U_i \cap U_j} s_{ui}^2} \cdot \sqrt{\sum_{u \in U_i \cap U_j} s_{uj}^2}}$$

Mean-Centered Rating Prediction

- The peer group $Q_i(u)$ of item v_i is defined as the top- k items that have high similarity with v_i , and have been rated by user u
- The weighted average of these ratings of the peer group is used as the predicted rating

$$\hat{r}_{ui} = \frac{\sum_{v_j \in Q_i(u)} \text{Adjusted Cosine}(v_i, v_j) \cdot r_{uj}}{\sum_{v_j \in Q_i(u)} |\text{Adjusted Cosine}(v_i, v_j)|}$$

- The basic idea is to leverage the **user's own ratings** of similar items in the final step of making a prediction

Mean-Centered Rating Prediction

- Example
 - Item similarity between v_1 and v_3

Table 2.2: Ratings matrix of Table 2.1 with mean-centering for adjusted cosine similarity computation among items. The adjusted cosine similarities of items 1 and 6 with other items are shown in the last two rows.

Item-Id $\Rightarrow$	1	2	3	4	5	6
User-Id $\Downarrow$						
1	1.5	0.5	1.5	-1.5	-0.5	-1.5
2	1.2	2.2	?	-0.8	-1.8	-0.8
3	?	1	1	-1	-1	?
4	-1.5	-0.5	-0.5	0.5	0.5	1.5
5	-1	?	-1	0	1	1
Cosine(1, j) (item-item)	1	0.735	0.912	-0.848	-0.813	-0.990
Cosine(6, j) (item-item)	-0.990	-0.622	-0.912	0.829	0.730	1

$$\text{Adjusted Cosine}(v_1, v_3) = \frac{1.5 * 1.5 + (-1.5) * (-0.5) + (-1) * (-1)}{\sqrt{1.5^2 + (-1.5)^2 + (-1)^2} \cdot \sqrt{1.5^2 + (-0.5)^2 + (-1)^2}} = 0.912$$

Mean-Centered Rating Prediction

- Example (cont'd)

- The rating prediction for v_1 by u_3 is based on the top-2 peer group, consisting of v_2 and v_3
- The rating prediction for v_6 by u_3 is based on the top-2 peer group, consisting of v_4 and v_5

Table 2.2: Ratings matrix of Table 2.1 with mean-centering for adjusted cosine similarity computation among items. The adjusted cosine similarities of items 1 and 6 with other items are shown in the last two rows.

Item-Id $\Rightarrow$	1	2	3	4	5	6
User-Id $\Downarrow$						
1	1.5	0.5	1.5	-1.5	-0.5	-1.5
2	1.2	2.2	?	-0.8	-1.8	-0.8
3	?	1	1	-1	-1	?
4	-1.5	-0.5	-0.5	0.5	0.5	1.5
5	-1	?	-1	0	1	1
Cosine(1, j) (item-item)	1	0.735	0.912	-0.848	-0.813	-0.990
Cosine(6, j) (item-item)	-0.990	-0.622	-0.912	0.829	0.730	1

$$\hat{r}_{31} = \frac{0.735 * 3 + 0.912 * 3}{0.735 + 0.912} = 3, \hat{r}_{36} = \frac{0.829 * 1 + 0.730 * 1}{0.829 + 0.730} = 1$$

Comparing User-based and Item-based Methods

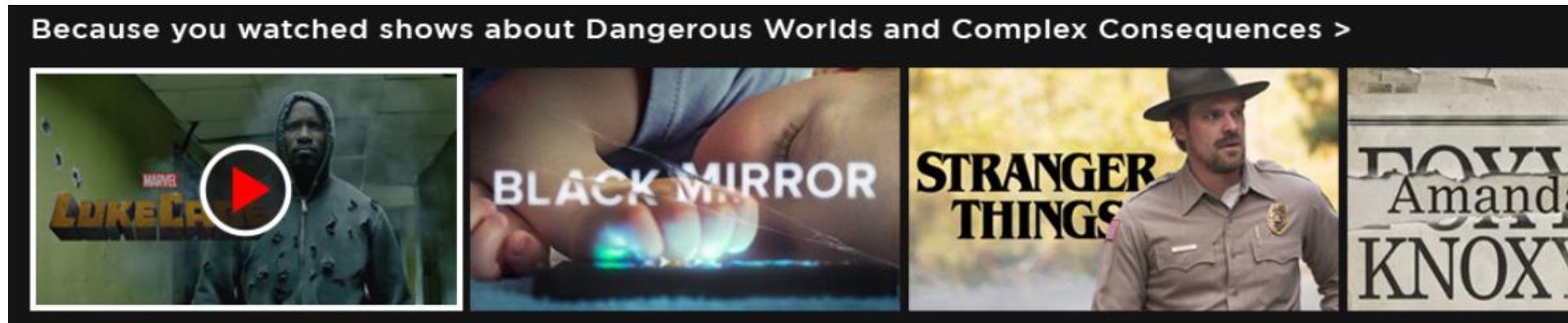
- **Advantages of Item-based Methods**

- They often provide more relevant recommendations because a **user's own ratings are used** to provide the recommendations
- In item-based methods, items similar to a target item are identified, and the user's own ratings for those items are used to extrapolate the rating for the target item
- For example, items similar to a target historical movie might be a set of other historical movies. In such cases, the user's ratings for the similar set may be highly indicative of their preference for the target movie

Comparing User-based and Item-based Methods

- **Advantages of Item-based Methods**

- Item-based methods can also provide a concrete rationale for the recommendation



- Such explanations can be effectively provided with item-based methods by leveraging item neighborhoods
- User-based methods cannot provide these explanations because the peer group cannot be directly used due to privacy concerns

Comparing User-based and Item-based Methods

- **Advantages of Item-based Methods**

- Item-based methods are more stable with respect to changes in ratings
- The number of users is generally much larger than the number of items. In such cases, two users may have a very small number of mutually rated items, whereas two items are more likely to have a larger number of users who have rated both
- In commercial systems, new users are likely to be added more frequently than new items. The computation of neighborhood items can be performed less frequently, as these item neighborhoods are unlikely to change significantly with the addition of new users

Comparing User-based and Item-based Methods

- **Advantages of User-based Methods**

- User-based methods may provide a more diverse range of items compared to item-based methods
- Greater diversity also encourages **serendipity**, allowing users to discover surprising and interesting items
- Item-based methods may sometimes recommend obvious items that are **not novel** compared to a user's previous experiences
- Without sufficient novelty, diversity, and serendipity, users may become bored with recommendations that are too similar to what they have already watched

Efficient Implementation and Computational Complexity

- The aforementioned discussion explains how to predict ratings for a particular user-item combination but does not address the actual ranking process
- A straightforward approach is to compute all possible rating predictions for the relevant user-item pairs and then rank these predictions
- While this is the basic approach used in current recommender systems, it is important to note that the prediction process for many user-item combinations reuses several intermediate quantities
- Therefore, it is advisable to have an offline phase to store these intermediate computations, which can then be leveraged during the ranking process

Efficient Implementation and Computational Complexity

- Neighbor-based methods are typically divided into an **offline** phase and an **online** phase
 - In the offline phase, user-user (or item-item) similarity values and the peer groups of users (or items) are computed
 - In the online phase, these similarity values and peer groups are leveraged to make predictions based on their relationships

Efficient Implementation and Computational Complexity

- **Offline phase**

- For n users and m items, each user has m' ratings ($m' \ll m$), and each item has n' ratings ($n' \ll n$)
 - To determine the peer groups of users and items, the computational complexities for each user and item are $O(m' \cdot n)$ and $O(n' \cdot m)$, respectively
 - In user-based methods, the offline running time for computing peer groups is $O(m' \cdot n^2)$
 - In item-based methods, the offline running time for computing peer groups is $O(n' \cdot m^2)$

- **Online phase**

- For a target user, the running time for rating predictions on m items is
 - $O(k \cdot m)$, where k is the size of the peer group

- **Comparing the running times of offline and online phases**

- $O(m' \cdot n^2)$ or $O(n' \cdot m^2) \gg O(k \cdot m)$

Strengths and Weaknesses of Neighborhood-based Methods

- **Strengths**

- Neighborhood-based methods are a simple and intuitive approach
- Easy to implement and debug
- Easy to justify why a specific item is recommended
- The interpretability of item-based methods is particularly notable
- The recommendations remain relatively stable with the addition of new items and users

- **Weaknesses**

- The offline phase can sometimes be impractical in large-scale settings
- This can sometimes be too slow or require too much space
- Limited coverage due to sparsity